

SOFTWARE DEVELOPMENT II (SD2) LAB PROJECT REPORT

Project Title: Interactive Flutter Portfolio Application

Student Name: Mohammad Fazle Rabbi
Institution: International Islamic University Chittagong (IIUC)
Department: Computer Science & Engineering
Course: Software Development II (SD2) Lab

1. EXECUTIVE SUMMARY

This report demonstrates how the Interactive Flutter Portfolio Application (`basic_portfolio`) encapsulates the practical software engineering principles, design patterns, and cross-platform mobile/web development methodologies taught in the Software Development II (SD2) Lab curriculum.

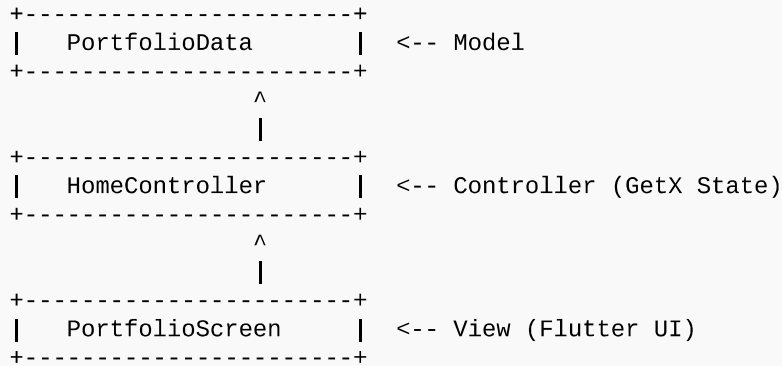
The application is built using Flutter and Dart, leveraging GetX for state management, reactive programming, and dependency injection. It incorporates custom responsive UI elements, a Midnight Cyber Glassmorphic visual theme, asset management, and cross-platform web/desktop compatibility.

2. KEY SD2 LAB LEARNING OUTCOMES & CODEBASE MAPPING

2.1 Model-View-Controller (MVC) Architectural Pattern

In SD2 Lab, a fundamental objective is learning to design modular, scalable applications by separating business logic from user interface presentation.

- **Model Layer (`portfolio_data.dart`):** Defines strongly-typed data structures (`PortfolioData`, `SocialLink`, `Project`) representing developer profile metrics, project lists, and skill arrays. This keeps the application data decoupled from UI widgets.
- **Controller Layer (`home_controller.dart`):** Extends `GetxController` to manage application state and animation triggers (`profileSize`, `profileColor`, `buttonScale`, `cardWidth`). It initializes portfolio data during `onInit()` and provides clean event handlers (`toggleProfileAnimation`, `toggleCardAnimation`, `toggleButtonAnimation`).
- **View Layer (`main.dart`):** Renders stateless widgets (`PortfolioScreen`) that consume controller states via `Obx` reactive observers, ensuring clean UI rendering without bloating widget trees.



2.2 Reactive State Management & Dependency Injection

A core requirement of SD2 Lab is managing dynamic application state efficiently without unnecessary full-screen rebuilds.

- **GetX Reactive Variables:** Observable properties (`.obs`) allow precise granular updates to specific subtrees when state mutates (e.g., `var profileSize = 110.0.obs;`).
- **Obx Observers:** Used in `main.dart` to rebuild only the animated profile ring and interactive components when state changes occur.
- **Dependency Binding:** Controller instances are injected lazily using `Get.put(HomeController())` within `BindingsBuilder` inside `MyApp`, practicing clean inversion of control.

2.3 Cross-Platform Web & Native Compatibility

Developing for multiple target platforms requires understanding platform constraints (e.g., browser sandboxing vs. native file system access).

- **Asset Packaging:** Standardized asset declaration in `pubspec.yaml` under `assets:` - `assets/images/` ensures assets are bundled correctly for both Web and Mobile/Desktop.
- **Platform-Agnostic Asset Resolution:** Refactored asset loading to use `AssetImage('assets/images/profile.jpg')` rather than native `dart:io File` calls, preventing runtime `Unsupported operation: _Namespace` exceptions in browser engines.

2.4 Modern UI/UX Design Systems & Ergonomics

SD2 Lab emphasizes human-computer interaction (HCI) principles, visual hierarchy, responsive layouts, and modern design aesthetic implementation.

- **Visual Hierarchy & Color Systems:** Implemented a Midnight Dark Cyber Glassmorphic theme using deep slate backgrounds (`#0F172A`), cyber cyan accents (`#06B6D4`), and violet gradients (`#8B5CF6`). Semi-transparent cards (`#1E293B`) with subtle border highlights (`#334155`) improve visual contrast and readability.
- **Button Placement & Call-To-Action (CTA) Strategy:** Primary high-intent actions ("Contact Me", "View Work") are positioned prominently in the Hero Header Section for immediate user access. Social

links are formatted as interactive pill badges with outward navigation indicators. Project repositories feature styled, dedicated "View Source" action buttons aligned to the lower-right of each card.

2.5 Asynchronous Programming & Third-Party Integration

SD2 Lab covers integrating external packages and handling non-blocking asynchronous operations.

- **External URL Launching:** Integrates `url_launcher` package with `async/await` control flows (`canLaunchUrl` & `launchUrl`) to launch social profiles, GitHub repositories, and `mailto:` email clients asynchronously.

2.6 Static Analysis & Software Quality Assurance (QA)

A major aspect of modern software engineering practice taught in SD2 is code quality and compliance with framework standards.

- **Linting & Analysis:** Used `flutter analyze` to eliminate compiler errors, enforce type safety, remove unused imports, and update deprecated APIs (e.g., replacing `.withOpacity()` with `.withValues(alpha: ...)`).

3. SD2 LAB LEARNING MATRIX

SD2 Lab Topic	Implementation in Project	Primary Source Files
Object-Oriented Data Modeling	Encapsulated data schemas (<code>PortfolioData</code> , <code>Project</code> , <code>SocialLink</code>)	<code>portfolio_data.dart</code>
State Management (GetX)	Reactive state variables (<code>.obs</code>) & controller lifecycle (<code>onInit</code>)	<code>home_controller.dart</code>
Declarative UI Construction	Widget composition (<code>Scaffold</code> , <code>Column</code> , <code>Wrap</code> , <code>Container</code> , <code>Obx</code>)	<code>main.dart</code>
Asset & Resource Management	Image asset bundling and <code>pubspec.yaml</code> registration	<code>pubspec.yaml</code>
Async Operations & Utilities	Asynchronous URL launching & email triggers	<code>main.dart</code>
UI/UX Ergonomics & Placement	Optimized CTA placement, dark theme design, glassmorphism	<code>main.dart</code>
Cross-Platform Compatibility	Web-safe rendering avoiding <code>dart:io</code> runtime errors	<code>main.dart</code>

4. CONCLUSION

This project serves as a practical implementation of the concepts taught in the SD2 Lab course. Through building this portfolio application, we demonstrated mastery in:

- Applying the MVC architectural pattern for clean code organization.
- Managing reactive state effectively with GetX.
- Designing modern, responsive, and ergonomic UI/UX experiences.
- Ensuring cross-platform web and mobile compatibility.
- Practicing static code analysis and adherence to Flutter software engineering standards.